Spring Boot 3.X 개발 심화

inky4832@daum.net

6. 고급 기능 및 운영

- 로깅과 모니터링
- 스케쥴링

6.1 로깅과 모니터링

로깅 (Logging)

- 개념
  - 프로그램 실행 중 발생하는 모든 사건(Event)을 기록하는 활동
  - 로그(Log)는 기록된 데이터를 의미

- 필요성

상황	설명
장애 발생	에러 지점/원인 즉시 확인
성능 문제	응답시간, 트랜잭션 분석 가능
보안 감시	의심 활동 추적 가능
운영 유지보수	근거 기반 대응 가능

6.1 로깅과 모니터링

System.out.println vs Logging

구분	System.out.println	Logging
목적	화면 출력	운영 로그
로그 레벨	지원 안됨	trace, debug, info, warn, error 등 5가지
파일 저장	지원 안됨	가능
성능	느림	고성능
필터링	지원 안됨	특정 Level만 출력 가능
운영 사용	비추천	필수

6.1 로깅과 모니터링

로깅 (Logging) 구성 요소

구성 요소	설명	예시
Logger	로그 생성 도구	Logback, Log4j2
Appender	로그 출력 위치 결정	Console, File, DB, Cloud
Layout	로그 메시지 포맷	%d{yyyy-MM}
Level	로그 심각도	trace, debug, info, warn, error

6.1 로깅과 모니터링

로그 레벨 (Level)

Level	설명	예시
ERROR	서비스 중단 급 문제	DB 장애
WARN	잠재적 문제	과도한 응답 지연
INFO	비즈니스 흐름 정보	로그인 성공
DEBUG	상세 디버깅 정보	파라미터 정보
TRACE	모든 내부 동작, 가장 상세	성능에 매우 민감

6.1 로깅과 모니터링

Java 로깅 프레임워크

구분	역할	현재 사용	비고
SLF4J	표준 API (로깅 추상화)	매우 높음	API 인터페이스 역할
Logback	**로깅 엔진 (로깅 구현체)**	**최고 수준**	**Spring Boot 기본**
Log4j2	로깅 엔진 (로깅 구현체)	조건부 사용	성능 민감 시스템 최적
Log4j(1.X)	구버전 로깅 엔진	사용 폐기	보안 취약점 발견
java.util.logging	JDK 기본 로깅	낮음	학습용 정도
Apache Commons Logging	과거 추상화 API	사용 감소	과거 Spring 기본, 현재 SLF4J로 대체

6.1 로깅과 모니터링

Logback

- 개념
  - Spring Boot에서 지원하는 기본 로그 생성 도구(Logger)
  - SLF4J API를 통해 호출되어 실제로 로그를 남기는 로깅 엔진

- 특징

특징	설명
고성능	빠른 처리, 비동기 로깅 지원
설정 유연	XML로 상세하게 제어 가능
프로파일 지원	dev/prod 환경에 따른 다른 설정
Rolling 파일 지원	날짜/용량 기준으로 파일 분할+보관
Console/File/DB 등 지원	Appender 확장성

6.1 로깅과 모니터링

패턴(Format) 이해

패턴지정 예시: %d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n

출력 예시: 2025-10-29 19:10:25.123 [http-nio-8080-exec-1] INFO com.example.UserService - User found: id=1

토큰	의미
%d{...}	시간 (포맷 지정)
%thread	실행 스레드
%-5level	로그 레벨 (좌측 정렬 5칸)
%logger{36}	로거명 (보통 클래스명, 최대36자)
%msg	메시지
%n	줄바꿈

6.1 로깅과 모니터링

Spring Boot + Logback 기본

- application.yml 설정

```
logging:
  level:
    root: INFO                                    # 전체 기본 레벨
    com.example.demo: DEBUG                       # 내 비즈니스 코드만 상세하게
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss.SSS} %-5level %logger{36} - %msg%n"
  file:
    name: c:₩₩temp₩₩app.log                      # 로그 파일 저장 경로
```

- 코드 설정

```
@Slf4j + log.trace("logger:{}", "TRACE")
         log.debug("logger:{}", "DEBUG")
         log.info("logger:{}", "INFO")
       log.warn("logger:{}", "WARN")
       log.error("logger:{}", "ERROR")
```

6.1 로깅과 모니터링

실습(Logback 기본)

```yaml
# application.yml
spring:
  application:
    name: my-app

server:
  port: 8090

logging:
  level:
    root: INFO                                  # 전체 기본 레벨
    org.springframework.web: TRACE
    com.example.demo: DEBUG                     # 내 비즈니스 코드만 상세하게
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss.SSS} %-5level %logger{36} - %msg%n"
  file:
    name: c:\\temp\\my-app.log                  # 로그 파일 저장 경로
```

6.1 로깅과 모니터링

실습(Logback 기본)

```java
// UserController.java
package com.example.demo.controller;

import org.springframework.web.bind.annotation.*;
import lombok.extern.slf4j.Slf4j;

@Slf4j
@RestController
public class UserController {

    @GetMapping("/hello-world")
    public String helloWorld() {

        log.trace("logger:{}", "TRACE");
        log.debug("logger:{}", "DEBUG");
        log.info("logger:{}", "INFO");
        log.warn("logger:{}", "WARN");
        log.error("logger:{}", "ERROR");

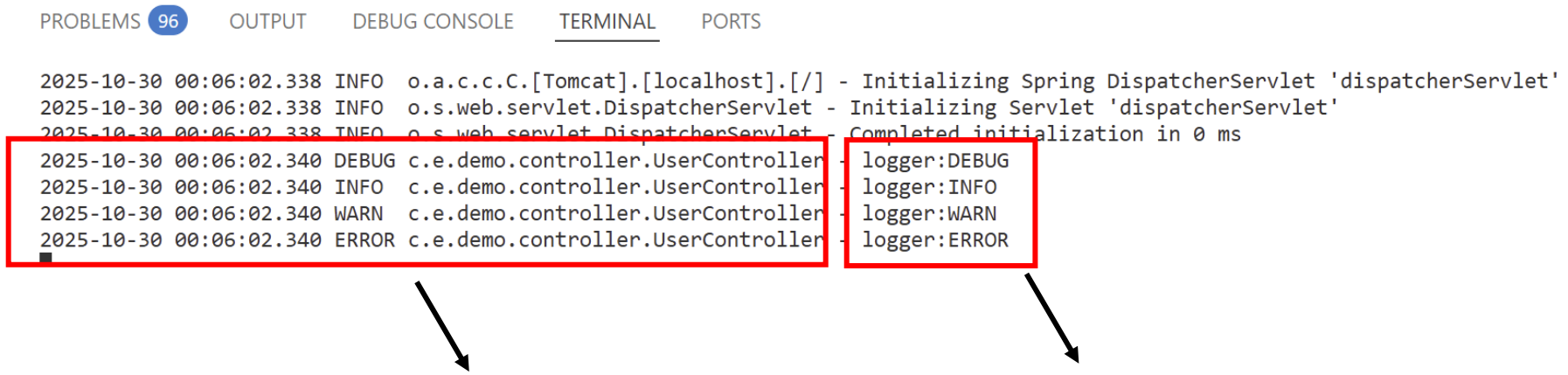
        return "Hello World";
    }

}
```

6.1 로깅과 모니터링

실습(Logback 기본)

GET http://localhost:8090/hello-world 요청.



"%d{yyyy-MM-dd HH:mm:ss.SSS} %-5level %logger{36} - %msg%n" Level : **DEBUG**

6.1 로깅과 모니터링

logback-spring.xml

- 개념
  - Spring Boot에서 Logback 설정을 확장하여 사용할 수 있는 전용 로깅 설정 파일

- 용도

목적	설명
고급 로깅 설정	파일 롤링, JSON 로그, 로그 전송, MDC, 비동기
운영/개발 분리	dev/prod 프로파일별로 파일/권한/레벨 분리 가능
중앙 로그 시스템 연동	Logstash, ELK, Datadog
성능 최적화	AsyncAppender로 I/O 부담 제거

6.1 로깅과 모니터링

logback-spring.xml

- 핵심 구성 요소

요소	설명
<property>	외부 경로/환경 값 변수화
<encoder>	로그 출력 형식 (패턴/JSON)
<appender>	로그 출력 대상 설정 (Console/File/Network)
<logger>	특정 패키지/클래스 레벨 설정
<root>	전체 기본 로거
<springProfile>	프로파일(dev/prod/test)별 설정 분리

6.1 로깅과 모니터링

logback-spring.xml 기본 구조

```
<configuration>

    <!-- 로그 공통 변수 -->
    <property name="LOG_HOME" value="./logs"/>

    <!-- 출력 형식 -->
    <encoder>
        <pattern>...</pattern>
    </encoder>

    <!-- 출력 대상 -->
    <appender name="CONSOLE"
class="...ConsoleAppender">
        ...
    </appender>

    <appender name="FILE" class="...RollingFileAppender">
        ...
    </appender>

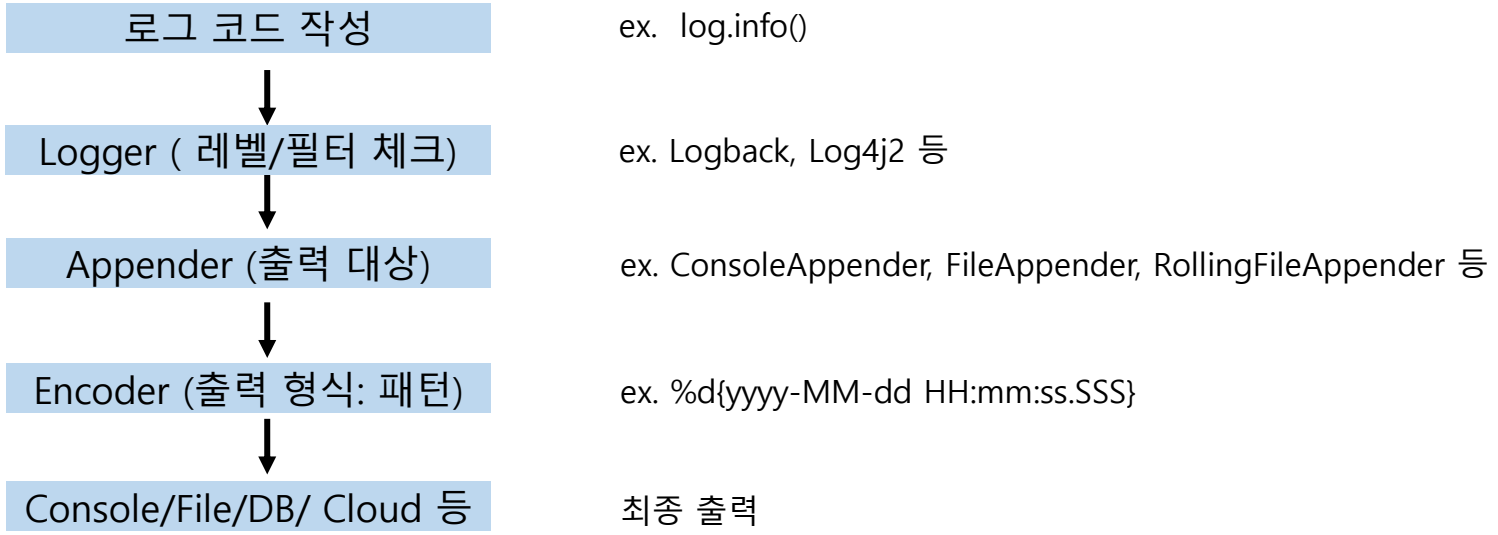
    <!-- 로거 설정 -->
    <logger name="com.example.demo"
level="DEBUG"/>

    <!-- 루트 로거 -->
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
        <appender-ref ref="FILE"/>
    </root>

</configuration>
```

6.1 로깅과 모니터링

로그 생성 흐름



6.1 로깅과 모니터링

실습(Logback-spring.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
   <!-- logback-spring.xml -->
<configuration scan="true">

    <!-- 공통 프로퍼티 -->
    <property name="LOG_HOME" value="${LOG_HOME:-./logs}"/>
    <property name="APP_NAME" value="${APP_NAME:-demo-app}"/>

    <!-- 콘솔 출력 (개발 친화 패턴) -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

 <!-- 파일 + 롤링(일/용량 기반) -->
    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>${LOG_HOME}/${APP_NAME}.log</file>
```

6.1 로깅과 모니터링

실습(Logback-spring.xml)

```
<!-- 날짜+용량 기준으로 파일 분할 -->
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <!-- 하루 단위로 새 파일 + 인덱스(용량 초과 시) -->
            <fileNamePattern>${LOG_HOME}/archive/${APP_NAME}.%d{yyyy-MM-dd}.%i.log.gz</fileNamePattern>

            <!-- 하루 안에서도 파일이 커지면 번호 증가(예: .0, .1...) -->
            <timeBasedFileNamingAndTriggeringPolicy
                    class="ch.qos.logback.core.rolling.SizeAndTimeBasedFNATP">
                <maxFileSize>100MB</maxFileSize>
            </timeBasedFileNamingAndTriggeringPolicy>

            <!-- 보관 정책 -->
            <maxHistory>14</maxHistory>          <!-- 14일 보관 -->
            <totalSizeCap>5GB</totalSizeCap>     <!-- 전체 합계 상한 -->
            <cleanHistoryOnStart>true</cleanHistoryOnStart>
        </rollingPolicy>
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>
```

6.1 로깅과 모니터링

실습(Logback-spring.xml)

```
    <!-- 로거 레벨(패키지별 조절) -->
    <logger name="org.springframework" level="INFO"/>
    <logger name="org.hibernate.SQL" level="WARN"/>
    <logger name="com.example.demo" level="INFO"/>

    <!-- 루트: 콘솔 + 파일 모두 출력 -->
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
        <appender-ref ref="FILE"/>
    </root>

</configuration>
```

6.1 로깅과 모니터링

실습(dev/prod 프로파일 분리)

```
# application.yml
server:
  port: 8090

# prod, dev 프로파일 공통 설정
spring:
  profiles:
    active: prod # 현재 활성화된 프로파일 설정 (dev 또는 prod)
```

6.1 로깅과 모니터링

실습(dev/prod 프로파일 분리)

```
<!-- logback-spring.xml -->
<configuration scan="true">
    <property name="LOG_HOME" value="${LOG_HOME:-./logs}"/>
    <property name="APP_NAME" value="${APP_NAME:-demo-app}"/>

    <!-- 공통 콘솔 -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{HH:mm:ss.SSS} %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>
<!-- 공통 파일(롤링) -->
    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>${LOG_HOME}/${APP_NAME}.log</file>
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <fileNamePattern>${LOG_HOME}/archive/${APP_NAME}.%d{yyyy-MM-dd}.%i.log.gz</fileNamePattern>
            <timeBasedFileNamingAndTriggeringPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedFNATP">
                <maxFileSize>100MB</maxFileSize>
            </timeBasedFileNamingAndTriggeringPolicy>
          <maxHistory>14</maxHistory>
            <totalSizeCap>5GB</totalSizeCap>
        </rollingPolicy>
```

6.1 로깅과 모니터링

실습(dev/prod 프로파일 분리)

```
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

    <!-- dev 전용: 내 코드 INFO, 하이버네이트 INFO -->
    <springProfile name="dev">
        <logger name="com.example.demo" level="INFO"/>
        <logger name="org.hibernate.SQL" level="INFO"/>
        <root level="INFO">
            <appender-ref ref="CONSOLE"/>
            <appender-ref ref="FILE"/>
        </root>
    </springProfile>
    <!-- prod 전용: 모두 WARN 이상, 콘솔+파일(롤링) -->
    <springProfile name="prod">
        <logger name="com.example.demo" level="WARN"/>
        <logger name="org.hibernate.SQL" level="WARN"/>
        <root level="INFO">
            <appender-ref ref="CONSOLE"/>
            <appender-ref ref="FILE"/>
        </root>
    </springProfile>

</configuration>
```

6.1 로깅과 모니터링

모니터링

- 개념
  - 시스템이 지금 어떤 상태인지 실시간으로 감시하는 것
    ex. Spring Boot Actuator

- 필요성

상황	설명
장애 발생	실시간 감지하고 조치
성능 저하	서버가 먼저 감지해서 대응
운영 효율	데이터 기반 의사결정

6.1 로깅과 모니터링

로깅 vs 모니터링

구분	로깅	모니터링
관점	사건(Event) 기록 중심	상태(State)/성능 지표 감시 중심
시간 축	사후 분석	실시간/준실시간 관찰
데이터 형태	텍스트/JSON	숫자 지표 ex. 카운트,평균 등
목적	정확한 원인 파악	조기 감지 및 건강도 파악
수집 단위	개별 요청/함수/예외 단위(세밀)	시스템/어플리케이션 전반, 엔드포인트 집계(거시)
대표 지표 항목	요청/응답 내용, 파라미터,예외	에러율/응답시간/CPU/메모리,GC
검색/분석 방법	키워드/필드 검색	대시보드/차트/임계치 기반 알림
Spring Boot 핵심	SLF4J + Logback, AOP 요청/응답 로깅	Actuator ex. health, metrics, loggers
장점	원인에 강함 (디버깅 필수)	추세/전반 건강도 파악에 강함

6.1 로깅과 모니터링

Actuator

- 개념
  - Spring Boot 어플리케이션의 운영, 모니터링, 진단을 위한 정보를 엔드포인트(Endpoint) 형태로 제공하는 기능
  - /actuator/... 주소를 통해 내부 상태를 확인 가능

    ex. /actuator/health, /actuator/metrics
  - **K8s, Prometheus 등에서 actuator가 제공한 metrics 정보를 사용**

- 주요 기능

기능	설명
모니터링과 제어	어플리케이션의 상태를 모니터링하고 필요시 제어 가능한 매커니즘 제공
시스템 상태보고	실시간으로 어플리케이션의 건강 상태, 매트릭, 환경정보 등을 제공
피드백 제공	시스템의 다양한 정보를 외부로 노출시켜 필요한 조치 가능하도록 지원
능동적 대응	단순한 모니터링을 넘어 로깅 레벨 변경, 상태확인과 같은 능동적인 작업 지원

6.1 로깅과 모니터링

Actuator

- 의존성 설정

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-actuator'
}
```

- 특징

항목	설명
어플리케이션 모니터링	어플리케이션의 상태, 건강 상태, 성능 지표 등을 실시간으로 모니터링
HTTP 엔드포인트	RESTful API 형태로 다양한 정보에 접근할 수 있는 엔드포인트 제공
운영 관리	로깅레벨 변경, 스레드 덤프 확인 등 운영 관련 작업을 수행 가능
보안 통합	Spring Security와 통합하여 엔드포인트에 대한 접근을 제어 가능

6.1 로깅과 모니터링

Endpoints

- 개념
  - Actuator가 제공하는 내부 관리용 URL 주소(API)
  - 어플리케이션의 상태, 환경, 설정, 로그, 매트릭 등을 외부에서 쉽게 확인하도록 지원
    ex. http://localhost:8080**/actuator/health**

- 특징
  - 각 Endpoint에 대해서 액세스(access)를 제어 가능
  - HTTP 또는 JMX를 통해 노출(expose)를 제어 가능
  - **Endpoint는 반드시 access가 허용되고 expose 되었을 때만 사용이 가능해짐**
  - 내장된 Endpoint는 사용 가능할 때만 자동으로 구성됨
  - 대부분 HTTP를 통해서 노출을 선택하며 경로는 /actuator/<endpoint ID> 형식을 취함

6.1 로깅과 모니터링

내장 Endpoints

https://docs.spring.io/spring-boot/reference/actuator/endpoints.html

Endpoint ID	설명	최종 Endpoint
health	어플리케이션의 전반적인 상태 정보 ex. UP, DOWN, OUT_OF_SERVICE	/actuator/health
info	어플리케이션 정보 표시 ex. 버전, 이름 등	/actuator/info
metrics	다양한 측정 지표 제공 ex. JVM, CPU, HTTP 요청 등	/actuator/metrics
beans	컨텍스트에 등록된 Spring Bean 목록 및 관계 정보 제공	/actuator/beans
env	어플리케이션 환경 설정 정보 제공	/actuator/env
mappings	요청 매핑 정보 ex. @RequestMappings	/actuator/mappings
loggers	로거 구성 및 레벨 조회/변경 기능 제공	/actuator/loggers
configprops	@ConfigurationProperties 목록 표시	/actuator/configprops
logfile	로그 파일 내용 액세스 logging.file.name 또는 logging.file.path 속성 설정 필요	/actuator/logfile

6.1 로깅과 모니터링

Liveness / Readiness

- 개념
  - SpringBoot 2.3 부터 K8s용으로 health 엔드포인트를 분리함

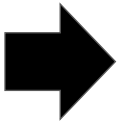
Endpoint	설명	설정값
/actuator/health/liveness	앱이 내부적으로 살아 있는지만 확인 ex. JVM 및 thread 정상	UP/DOWN
/actuator/health/readiness	서비스 가능한 상태만 확인 ex. DB 연결, Redis 연결, 외부 의존성 OK	UP/DOWN

6.1 로깅과 모니터링

Liveness / Readiness

- 설정

```
management:
  endpoint:
    health:
      probes:
        enabled: true
      show-details: always
  endpoints:
    web:
      exposure:
        include: health,info
```



활성화
/actuator/health/liveness
/actuator/health/readiness

6.1 로깅과 모니터링

Liveness / Readiness

- K8s 의 Deployment 예시

```
startupProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  failureThreshold: 30
  periodSeconds: 10

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 20
```

```
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 20
  periodSeconds: 10
```

6.1 로깅과 모니터링

Endpoint에 대한 expose

- 개념
  - Actuator의 Endpoint를 HTTP 요청으로 접근할 수 있게 노출시키는 설정
  - 보안을 위해서 /actuator/health 만 노출되어 있고 거의 대부분 숨겨둠

- 설정 방법
  - 각각의 property는 포함하거나 제외할 endpoint 목록을 지정
  - "*" 지정하면 모든 endpoint를 대상으로 함
  - include와 exclude에 설정된 내용이 상충하면 exclude에 대한 내용이 우선 순위가 높음

property	설명	기본값
management.endpoint.web.exposure.include	외부에서 접근 가능한 목록 지정	health
management.endpoint.web.exposure.exclude	노출하지 않을 목록 지정	

6.1 로깅과 모니터링

expose 예

```
# application.yml

management:
  endpoints:
    web:
      exposure:
        include: "*"
        exclude:
          - env
          - metrics
```

위 경우는 evn와 metrics 를 제외한 모든 Endpoint가 expose 됨

6.1 로깅과 모니터링

Endpoint에 대한 access

- 개념
  - 노출(expose)된 Actuator Endpoint에 대해 얼마나 접근을 허용할지 정하는 접근 수준
  - 기본적으로 shutdown 과 heapdump를 제외한 나머지는 바로 접근이 가능
  - **none(차단) / read-only(읽기만) / unrestricted (제한 없음) 3가지 중에서 사용**
    (deprecated된 기존 방식: enabled=true|false)

- 평가 순서
  - 먼저 노출(expose)이 되어야 하고 그 다음 access가 적용됨
  - 노출이 안되면 access 를 풀어도 외부에서 접근 불가

6.1 로깅과 모니터링

Endpoint에 대한 access

- 설정 방법

property	설명
management.endpoints.access.default	기본값. 어떤 endpoint도 별도 설정이 없을 때 적용 none|read-only|unrestricted 중 하나 지정
management.endpoints.access.max-permitted	상한선. 어떠한 경우에도 이 수준을 넘을 수 없음
management.endpoint.**<ID>**.access	개별적인 endpoint 제어 방법 ex. management.endpoint.**info**.access=read-only

6.1 로깅과 모니터링

전역 차단(옵트인) 예

```
management:
  endpoints:
    access:
      default: none
    web:
      exposure:
        include: health,info,metrics
  endpoint:
    health:
      access: read-only
    info:
      access: read-only
    metrics:
      access: read-only
```

6.1 로깅과 모니터링

Sanitize (민감한 마스킹)

- 개념
  - Endpoint가 어플리케이션 설정 정보를 노출할 때 비밀번호/토큰 등 민감한 값을 자동으로 *** 로 대체(masking)하는 기능

    ex. /actuator/env 와 /actuator/configprops 등

- 제어 방법

property	값
management.endpoint.<ID>.show-values	never (*** 대체) always (항상 실제값 표시) when-authorized (권한이 있으면 실제값 표시)

수고하셨습니다.